



UNIVERSITATEA DIN BUCUREȘTI

**FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ**



SPECIALIZAREA CALCULATOARE SI TEHNOLOGIA INFORMATIEI

Proiect la Metode de Dezvoltare Software

Smart Hiring Career Matching

Studenti:

**Andronache Daniel,
Mihai Cristian Ciobanu,
Rebleanu Robert Daniel**

Iunie 2026

I. Descriere

Prezentul proiect propune dezvoltarea unui portal de recrutare inteligent (smart hiring platform), care integrează tehnici de career matching asistat de inteligență artificială, cu scopul de a optimiza procesul de selecție a candidaților, oferind în același timp suport personalizat fiecărei părți implicate.

Platforma se bazează pe doi agenți AI complementari care estimează cât de bine se potrivește CV-ul unui candidat cu descrierea jobului la care acesta aplică:

- **Agentul destinat angajatorilor**, care oferă un scor de compatibilitate fiecărui candidat care aplică la jobul curent, ajutând la selecția candidaților.
- **Agentul destinat candidaților**, care evaluează compatibilitatea dintre CV-ul utilizatorului și descrierea joburilor la care vrea să aplice, menit să ghideze candidatul în identificarea oportunităților celor mai relevante pentru profilul său profesional.

II. Arhitectura



Sistemul este conceput pe baza unei **arhitecturi de tip client-server**, structurată în patru componente principale: frontend, backend, baza de date și agenții AI. Limbajul de programare principal este **Java**.

1. Frontend + Backend

Framework utilizat – Spring Boot, implementat cu:

- Spring Data JPA pentru lucrul cu baze de date
- Spring Security pentru autentificare
- Spring Web pentru implementarea MVC
- Tymeleaf – pentru frontend

2. Baza de date

Baza de date PostgreSQL, implementată prin imagine docker, folosind docker-compose pentru portabilitate.

3. Descrierea modelului

Entitățile modelului datelor sunt următoarele: Useri (Candidati, Angajatori, Admin), CV, Aplicări, Anunțuri, Companii.

- **User** – interfață pentru implementarea celor 3 tipuri de utilizatori. Are atributele rol, email, parola, data înființare cont.
- **Candidat** – implementarea candidatului (rol = Candidat)
- **Angajator** – implementarea angajatorului (rol = Angajator)
- **Admin** – implementare administrator (rol = Admin)
- **CV** – salvează CV-urile candidaților în format binar
- **Aplicare** – descrie relația dintre Candidat și Anunț, gestionează aplicarea candidatului la un anunț publicat pe platformă
- **Anunt** – oferta de job publicată de către Angajator, are atributele titlu, descriere, domeniu, salariu minim, salariu maxim, experiența necesară, timp de lucru.
- **Companie** – compania din care face parte un angajator, conține informații despre tipul companiei și locație.

4. Agenți AI

Framework – Spring AI

Implementare prin apeluri API folosind modelele opensource de la GroqCloud.

III. Procesul de dezvoltare

1. Backlog-uri

Managementul cerințelor proiectului a fost realizat folosind platforma Jira.

☐	▼ KAN-7 Functionalitati de baza	Unassigned	Daniel Andronac...	= Medium	TO DO ▼
☐	KAN-10 Angajator, vreau sa postez anunturi in cadr...	Daniel Andronache	Daniel Andronac...	= Medium	IN CURS ▼
☐	KAN-9 Candidat, vreau sa accesez anunturile curente	omega_329	Daniel Andronac...	= Medium	DONE ▼
☐	KAN-8 Ca utilizator, vreau sa ma autentific in cont	Robert Daniel Rebleanu	Daniel Andronac...	= Medium	IN CURS ▼
☐	▼ KAN-3 Backend si Baza de Date	Unassigned	Robert Daniel R...	= Medium	DONE ▼
☐	☒ KAN-6 Creare clase Entitate Repository si Controller	Robert Daniel Rebleanu	Robert Daniel R...	= Medium	DONE ▼
☐	☒ KAN-5 Diagrama ER	omega_329	Robert Daniel R...	= Medium	DONE ▼
☐	☒ KAN-4 Configurare PostgreSQL si configurare Sprin...	Daniel Andronache	Robert Daniel R...	= Medium	DONE ▼

Fig 1.1

☐	☒ KAN-18 Unit Testing cu Junit	omega_329	Daniel Andronac...	= Medium	TO DO ▼	Unre
☐	▼ ☒ KAN-15 Implementarea de agenti AI	Unassigned	Daniel Andronac...	= Medium	TO DO ▼	Unre
☐	KAN-17 Agentul candidatului pentru recomandarea j...	Robert Daniel Rebleanu	Daniel Andronac...	= Medium	TO DO ▼	Unre
☐	KAN-16 Agentul angajatorului pentru evaluarea can...	Daniel Andronache	Daniel Andronac...	= Medium	TO DO ▼	Unre
☐	▼ ☒ KAN-12 Aplicarea la joburi si incarcarea de CV-uri	Unassigned	Daniel Andronac...	= Medium	TO DO ▼	Unre
☐	KAN-14 Implementarea logicii de aplicare la joburi s...	Daniel Andronache	Daniel Andronac...	= Medium	TO DO ▼	Unre
☐	KAN-13 Creare pagina de profil pentru a incarca CV...	omega_329	Daniel Andronac...	= Medium	TO DO ▼	Unre
☐	☒ KAN-11 Initializare frontend - pagina principala - tyme...	omega_329	Daniel Andronac...	= Medium	DONE ▼	Tern

Fig 1.2

2. Diagrame

- Diagrama conceptuală a bazei de date

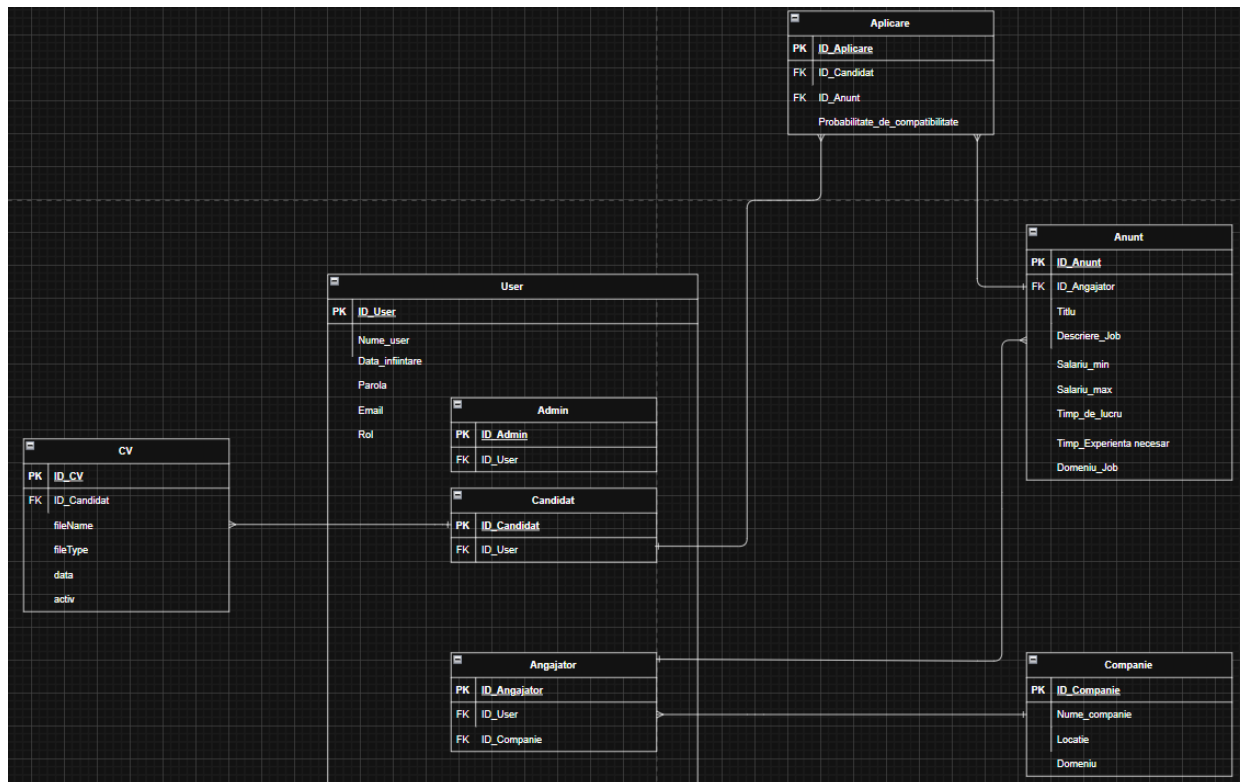


Fig 2.1

- Diagrama UML

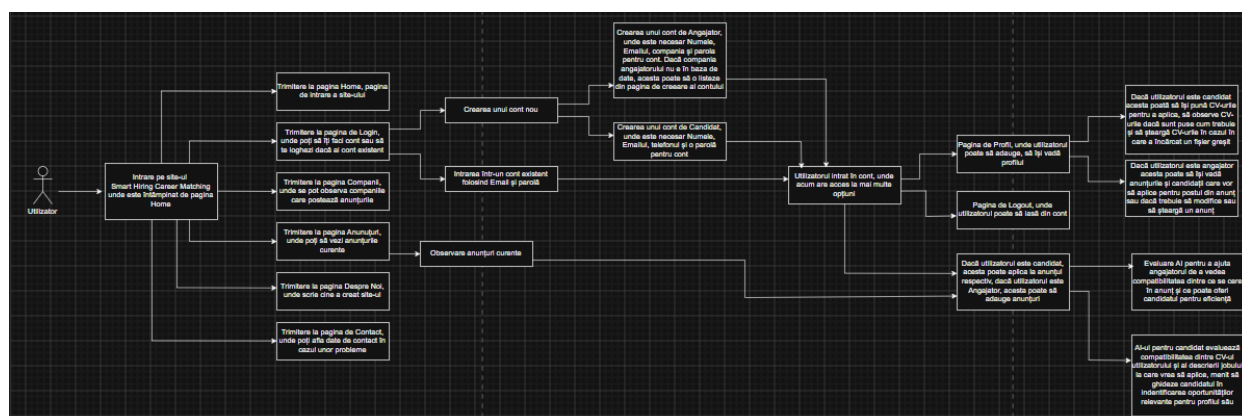


Fig 2.2

3. Design patterns

Singleton

În cadrul acestui proiect, pattern-ul Singleton nu a fost implementat manual, ci a fost utilizat implicit, prin mecanismul de gestionare a bean-urilor oferit de Spring Boot. Principalele clase Singleton utilizate sunt de 3 tipuri:

1. Controller – clase care gestionează interacțiunea dintre backend și frontend, prin expunerea de endpoint-uri REST și procesarea request-urilor/răspunsurilor HTTP.

Cod (Java):

```
@Controller
@RequestMapping("/anunturi")
public class AnuntController {

    //implementare controller
}
```

2. Repository – clase responsabile de accesul la baza de date, realizând operațiile CRUD (Create, Read, Update, Delete) asupra entităților, prin intermediul Spring Data JPA.

Cod (Java):

```
public interface AnuntRepository extends
CrudRepository<Anunt,Integer> {
    List<Anunt>findByAngajator(Angajator angajator);
}
```

3. Service – clase care implementează logica de business a aplicației, fiind apelate de către Controller și utilizând Repository pentru accesul la date.

Cod(Java):

```
@Service
public class AnunturiService {
    private final AnuntRepository anuntRepository;
    public AnunturiService(AnuntRepository anuntRepository){
        this.anuntRepository = anuntRepository;
    }
    //implementare servicii pentru anunturi
}
```

MVC (MODEL – VIEW – CONTROLLER)

Utilizarea design pattern-ului MVC a separat aplicația în 3 componente distincte:

1. Model – datele și logica aplicației, implementat prin clase **Entity**, mapate pe tabele din baza de date prin Spring Data JPA. Prin intermediul acestor clase, aplicația crează automat tabelele din baza de date.

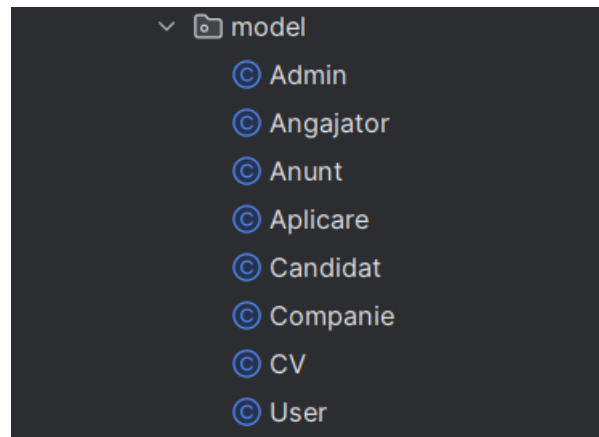


Fig 3.1. Structura fișierelor java din folderul „model” al aplicației

2. Controller – implementat prin clase Controller

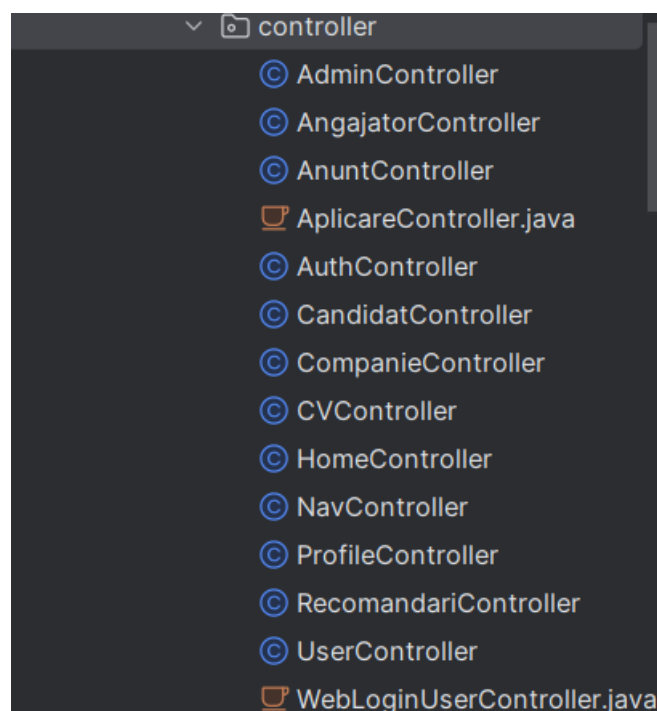


Fig 3.2. Structura fișierelor java din folderul „controller” al aplicației

3. View – implementat prin Tymeleaf framework. Fișierele html sunt randate direct de către clasele controller și primesc obiecte Model (obiecte Spring care transportă date din controller către view). Fișierele html sunt gasite in folderul de resurse („resources”).

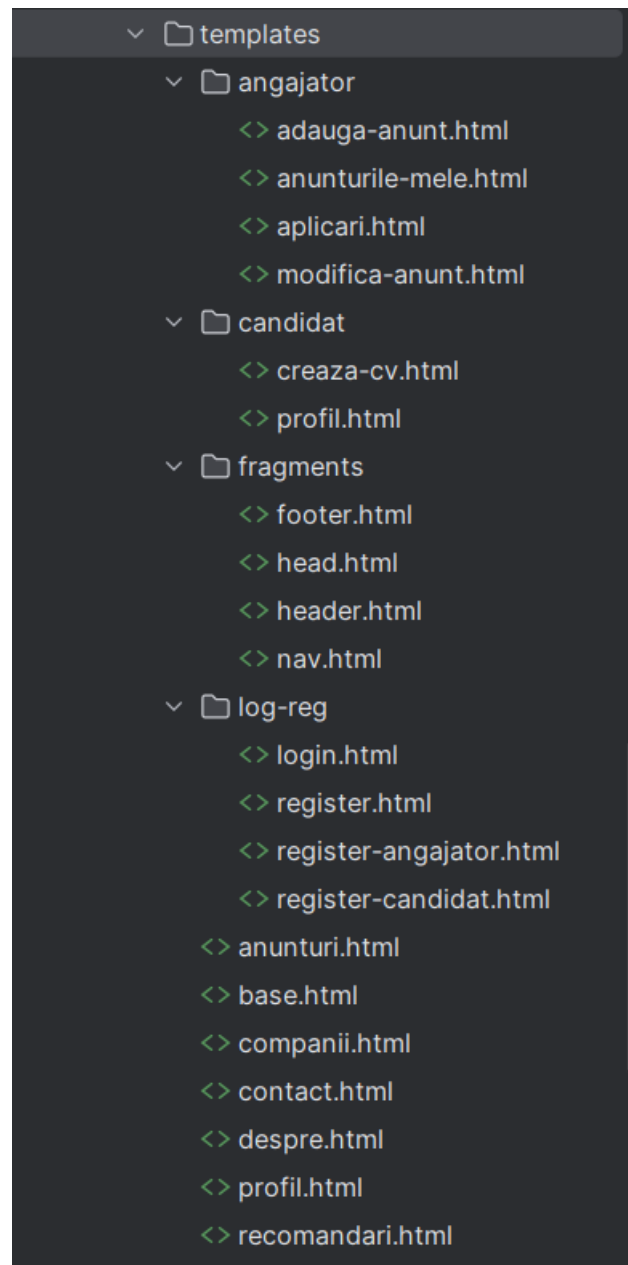


Fig 3.3. Structura fișierelor html din folderul „resources”

4. Implementarea de agenți AI

Agentii AI au fost implementați cu ajutorul Spring AI, un modul oficial Spring care permite interacțiunea cu modele AI prin intermediul interfeței ChatClient . Pentru extragerea textului din CV, a fost utilizată librăria pdfbox pentru extragerea din PDF-uri și Tess4J ca alternativă.

I. Agent AI pentru evaluarea candidaților

Acest agent a fost implementat prin intermediul clasei AgentAngajator.java. Agentul primește textul CV-ului candidatului și descrierea jobului, construiește un prompt structurat (system + user) și transmite cererea către modelul AI configurat, returnând un scor de compatibilitate (0-100) împreună cu o explicație succintă.

Utilizare în aplicație:

Cand Angajatorul verifică aplicările pentru un anunț propriu, acesta are la dispoziție un buton care oferă evaluarea AI în dreptul fiecărui aplicant.

II. Agent AI pentru recomandarea joburilor

Acest agent a fost implementat prin intermediul clasei AIService.java. Agentul este responsabil de calcularea unui scor numeric de compatibilitate (0.0–100.0) între CV-ul candidatului și un anunț de job. Pentru a asigura un format de răspuns exploatabil, prompt-ul transmis modelului impune explicit returnarea unui singur număr, fără text adițional. Pentru a asigura corectitudinea acestui proces, scorul este extras prin intermediul unei expresii regulate, cu o valoare implicită de siguranță 0.0.

Utilizare în aplicație:

Scorul de compatibilitate este folosit pentru determinarea unei recomandări de joburi pentru CV-ul analizat de către agent. Aplicația va selecta un top 3 anunțuri la apăsarea butonului corespunzător din bara de navigație.